NIM – NAMA :

- 2802507562 – Andreas Tri Dewa Putra
- 2802512865 – Dhammika Kumara
- 2802512530 – Limanto Winata HM

Kelompok : Ga ADA / Group-4

Kelas : LF-01

Mata Kuliah : Algorithm Design & Methods

Soal : AOL

A. Analisis kode A

SOAL:



Problem A

National Science Olympiad

In the recent National Science Olympiad, a new tie-breaker rule was implemented using time penalties measured in minutes.

The olympiad ran for two days and participated in by N contestants. The i-th participant, whose name is S_i, received A_i points with time penalty B_i on the first day, and received C_i points with time penalty D_i on the second day.

The contestants are ranked based on their total points from both days, with **higher points** being ranked higher. Contestants who receive the same total points will be ranked based on the sum of their time penalties from both days, with **lower penalties** being ranked higher. If there are still ties, they will be ranked based on their name, with the **lexicographically smaller name** being ranked higher.

Your task is to output the name of the contestants ranked from highest to lowest.

Input

The first line contains an integer N ($1 \leq N \leq 100$). Each of the next N lines contains a string S_i ($1 \leq |S_i| \leq 10$), containing uppercase English alphabets, followed by four integers A_i, B_i, C_i, and D_i ($0 \leq A_i, B_i, C_i, D_i \leq 300$) representing their points and penalties from both days.

Output

Output N lines, each containing the name of the contestants, in order from the higher-ranked to the lower-ranked.

Sample Input 1	Sample Output 1
5 ANDI 200 120 150 130 BUDI 170 70 180 170 CUPU 0 300 0 300 DEWA 300 0 300 0 MALANG 0 300 0 300	DEWA BUDI ANDI CUPU MALANG

Explanation of Sample 1: Let us consider the ranks between ANDI and BUDI. Both ANDI and BUDI received

Soal meminta sebuah rancangan kode, yang dimana akan diberikan sebuah inputan untuk pengulangan sebanyak n kali, lalu akan di input sebuah string, dan juga 4 buat

integer yang akan menjadi patokan untuk menentukan skor dan akan menjadi penentu nilai akhir tiap player. Inti dari soal adalah membuat sebuah sorting, yang dimana kode akan mengurutkan urutan dari player yang memiliki skor tertinggi hingga ke skor terendah. Dan ada kondisi khusus tersendiri, jika skor dari pemain sama, maka akan diurutkan berdasarkan nama

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(){
    char name[100][11];
    int score1;
    int time1;
    int score2;
    int time2;
    int totalScore[100];
    int totalTime[100];
    int n;
    scanf("%d", &n);
    for(int i = 0; i < n; i++){
        scanf("%s", name[i]);
        getchar();
        scanf("%d %d %d %d", &score1, &time1, &score2, &time2);
        totalScore[i] = score1 + score2;
        totalTime[i] = time1 +  time2;
    }
    for(int i = 0; i < n - 1; i++){
        for(int j = 0; j < n - 1; j++){
            int swap = 0;
            if (totalScore[j] < totalScore[j+1]) swap = 1;
            else if (totalScore[j] == totalScore[j+1] && totalTime[j] >
totalTime[j+1]) swap = 1;

            if(swap){
                int temp = totalScore[j];
                totalScore[j] = totalScore[j+1];
                totalScore[j+1] = temp;
                temp = totalTime[j];
                totalTime[j] = totalTime[j+1];
```

```
                totalTime[j + 1] = temp;
                char tempName[100];
                strcpy(tempName, name[j]);
                strcpy(name[j], name[j+1]);
                strcpy(name[j+1], tempName);
            }
        }
    }
    for(int i = 0; i < n; i++){
        printf("%s\n", name[i]);
    }
}
```

ANALISA:
Cara yang saya gunakan untuk menyelesaikan soal A, yang dimana di dalam soal saya diminta untuk membuat sebuah sorting ranking dari suatu integer, dan jika integer sama maka akan menggunakan string dari nama. Untuk menyelesaikan soal, maka solusi yang saya gunakan adalah dengan melakukan merge sort. Alasan penggunaan merge sort sebagai solusi adalah, panjang dari String yang tidak terlalu memakan memory dan juga, untuk mendapat solusi dengan waktu yang singkat. Maka, solusi yang saya pilih sebagai jawaban dari soal A, adalah bubble sort.

Dari solusi yang dihasilkan dengan menggunakan bubble sort, saya mendapat time complexity sebanyak O(n^2), yang dimana untuk soal itu cukup, tapi itu masih bukanlah solusi yang maksimal.

Solusi yang lebih baik:
Untuk solusi yang lebih baik lagi, bisa menggunakan sort seperti qsort yang mampu mereduksi kodingan lebih cepat, sehingga time complexity bisa direduksi menjadi O(n log n), karena, qsort lebih optimal dalam melakukan sorting dan mampu memangkas waktu dengan lebih ringkas.

B. ANALISA KODE K

SOAL:



international collegiate programming contest
INDONESIA NATIONAL CONTEST
INC 2025



Problem K

Palindrome Partition

A palindrome is a string that reads the same forwards as backwards. For example, `radar`, `noon`, and `a` are palindromes, while `bathtub`, `thought`, and `is` are not.

Given a string S consisting only of lowercase English letters. You can rearrange the letters in S in any order you like. Your task is to split S after rearrangement into as few palindromic substrings as possible.

Input

The first and only line contains the string S ($1 \leq |S| \leq 200\,000$) containing lowercase English letters.

Output

On the first line, print the minimum number k of palindrome substrings.

On the next k lines, print the palindromes that the string S (after rearrangement) can be split into. If there are multiple ways to split, you may output any of them.

Sample Input 1	Sample Output 1
larcevalecer	2 level racecar

Explanation of Sample 1: We can rearrange the input string into `levelracecar`, then split the string into two palindromes: `level` and `racecar`. No rearrangement can produce a single palindrome, so the minimum number of palindromes is 2.

Sample Input 2	Sample Output 2
abab	1 baab

Soal ini membahas tentang konsep palindrom, yaitu string yang jika dibaca dari depan maupun dari belakang tetap sama. Diberikan sebuah string S yang hanya terdiri dari huruf kecil alfabet Inggris dengan panjang hingga 200.000 karakter. Yang bikin soal ini beda adalah kita bebas mengacak urutan huruf dalam string tersebut, jadi urutan awal string tidak mengikat sama sekali.

Setelah huruf-huruf dalam string diatur ulang dengan cara apa pun, tugas utama yang diminta adalah membagi string hasil pengacakan tersebut menjadi beberapa substring, di mana setiap substring harus berupa palindrom. Dari semua kemungkinan pengacakan dan pembagian yang

bisa dilakukan, kita diminta untuk mencari jumlah minimum substring palindrom yang bisa dibentuk.

Output yang diharapkan terdiri dari dua bagian. Pertama, kita harus mencetak sebuah bilangan bulat K yang menyatakan jumlah minimum substring palindrom yang diperlukan. Kedua, kita harus mencetak K baris berikutnya yang masing-masing berisi substring palindrom hasil pembagian string tersebut. Jika terdapat lebih dari satu solusi yang memenuhi jumlah minimum, maka solusi mana pun boleh ditampilkan.

Intinya, soal ini tidak meminta kita memeriksa palindrom dari urutan asli string, melainkan menuntut kita untuk berpikir secara optimal dalam mengelompokkan huruf-huruf agar bisa membentuk palindrom sebanyak mungkin dengan jumlah potongan sesedikit mungkin. Fokus utama permasalahan ini ada pada pengaturan frekuensi karakter dan bagaimana karakter-karakter tersebut dapat disusun agar memenuhi sifat palindrom secara efisien.

CODE :

```c
#include<stdio.h>
#include<string.h>

int main(){
    char kata[200001]; // O(1) space
    int alphabet[26] = {0};
    int counter = 0;

    // Kompleksitas Waktu: O(N), di mana N adalah panjang string
    // Kompleksitas Ruang: O(N) untuk menyimpan input
    scanf("%s", kata);
    int length = strlen(kata);

    // O(N) - Menghitung frekuensi karakter
    for(int i = 0 ; i<length; i++){
        alphabet[kata[i] - 'a']++;
    }

    // O(26) -> O(1) - Menghitung karakter yang jumlahnya ganjil
    for(int i = 0 ; i<26; i++){
        if(alphabet[i] % 2 != 0){
```

```
            counter++;
        }
    }

    // Logika penentuan jumlah palindrom
    if(counter == 0) counter = 1;
    printf("%d\n", counter); // Output jumlah palindrom minimum

    char oddChars[26];
    int oddCount = 0;
    // O(1) - Mengambil satu karakter ganjil untuk jadi 'tengah'
palindrom
    for(int i = 0; i < 26; i++){
        if(alphabet[i] % 2 != 0){
            oddChars[oddCount++] = 'a' + i;
            alphabet[i]--;
        }
    }

    /* KRITIK KERAS:
       Bagian di bawah ini adalah O(N) secara logika,
       TAPI 'char Parts[counter][100001]' sangat berbahaya.
       Jika counter = 200.000, maka 200.000 * 100.000 = 20 GB memori.
       Ini bakal STACK OVERFLOW / Memory Limit Exceeded!
    */
    char Parts[counter][100001];
    int PL[counter];
    for(int i = 0; i < counter; i++){
        PL[i] = 0;
        Parts[i][0] = '\0';
    }
```

```
    // O(N) - Mendistribusikan pasangan karakter ke tiap bagian
palindrom
    int curr = 0;
    for(int i = 25; i >= 0; i--){
        while(alphabet[i] > 0){
            Parts[curr][PL[curr]++] = 'a' + i;
            alphabet[i] -= 2;
            curr = (curr + 1) % counter;
        }
    }

    // O(N) Mencetak hasil akhir
    for(int k = 0; k < counter; k++){
        // Cetak setengah bagian awal
        for(int i = 0; i < PL[k]; i++)   printf("%c", Parts[k][i]);
        // Cetak karakter tengah (jika ada)
        if(k < oddCount) printf("%c", oddChars[k]);
        // Cetak setengah bagian akhir (mirror)
        for(int i = PL[k] - 1; i >= 0; i--) printf("%c",
Parts[k][i]);
        printf("\n");
    }

    return 0;
}

/*
   TOTAL COMPLEXITY:
   Time Complexity: O(N)
   Space Complexity: O(N * counter) -> Secara teoritis buruk (bisa
O(N^2))
*/
```

TOTAL COMPLEXITY:

Time Complexity: O(N)

Space Complexity: O(N * counter)

PENJELASAN:

Dalam menyelesaikan Problem K: Palindrome Partition pada kompetisi INC 2025 , saya fokus pada optimalisasi waktu karena batasan panjang string S mencapai 200.000 karakter. Strategi utama saya adalah memanfaatkan karakteristik dasar palindrom, di mana sebuah palindrom hanya dapat memiliki maksimal satu jenis karakter dengan frekuensi ganjil sebagai titik tengahnya. Oleh karena itu, saya menghitung jumlah karakter yang berfrekuensi ganjil (counter) untuk menentukan jumlah minimal substring palindrom yang harus dibentuk. Jika tidak ada karakter ganjil, saya menetapkan minimal satu palindrom untuk memenuhi syarat *output*.

Saya memilih untuk mendistribusikan pasangan karakter genap secara merata ke seluruh slot palindrom yang tersedia menggunakan metode *round-robin*. Hal ini saya lakukan agar panjang setiap palindrom relatif seimbang dan struktur simetrisnya tetap terjaga saat dicetak. Penggunaan array dua dimensi Parts dan array oddChars memungkinkan saya menyusun setiap bagian palindrom secara efisien dalam satu kali proses iterasi, sehingga total kompleksitas waktu algoritma ini adalah linear O(N). Angka ini sangat aman untuk melewati batas waktu (Time Limit) yang ketat dalam standar kompetisi ICPC.

Meskipun pendekatan ini sangat cepat secara eksekusi, saya menyadari adanya risiko pada manajemen memori *stack* akibat penggunaan *Variable Length Array* (VLA) pada Parts[counter][100001]. Dalam skenario lomba ini, penggunaan tersebut masih aman karena jumlah baris (counter) dibatasi oleh jumlah alfabet (26 karakter), namun untuk pengembangan lebih lanjut atau jika menghadapi batasan karakter unik yang lebih besar, saya akan mempertimbangkan penggunaan memori global atau alokasi dinamis guna menghindari risiko *stack overflow*. Secara keseluruhan, kode ini dirancang untuk memprioritaskan kecepatan penulisan (*coding speed*) dan efisiensi waktu eksekusi yang optimal sesuai dengan kebutuhan situasi lomba.

C. ANALISA KODE C

SOAL:



international collegiate programming contest
INDONESIA NATIONAL CONTEST
INC 2025



Problem C

Polynomially Constructed

You are given an integer N and an array of **integers** $[y_1, y_2, \ldots, y_N]$ of size N.

You want to construct the following:

- a polynomial $P(x) = c_0x^0 + c_1x^1 + \ldots + c_dx^d$ where the coefficients are **real numbers**, and for some *degree* d;
- an array of **real numbers** $[x_1, x_2, \ldots, x_N]$ of size N such that $x_1 < x_2 < \ldots < x_N$;

and the construction satisfies $P(x_1) = y_1$, $P(x_2) = y_2$, ..., $P(x_N) = y_N$.

It can be proven that such a construction always exists. Your task is to find the minimum polynomial degree d that you can construct.

Input

The first line contains an integer N ($1 \leq N \leq 100\,000$). The second line contains N integers representing y_i ($-10^9 \leq y_i \leq 10^9$).

Output

Output the minimum degree d in a single line.

Sample Input 1	Sample Output 1
5 2 -1 1 1 0	3

Explanation of Sample 1: We can construct a polynomial $\frac{29}{3}x^0 + 8x^1 + 0x^2 - \frac{2}{3}x^3$, and array $[-2.81273..., -2, -1.24361..., 3.86937..., 3.91423...]$.

Sample Input 2	Sample Output 2
5 1 0 0 0 1	4

Soal ini membahas mengenai fungsi polinomial dan menanyakan derajat polinomial terkecil yang dapat digunakan untuk memenuhi semua nilai-y yang diminta oleh user. Ini dapat dikerjakan dengan mengertikan ciri-ciri sebuah fungsi. Fungsi yang sedang naik tidak akan mencapai nilai-y yang lebih kecil atau sama dengan nilai-y yang sudah dilewati tanpa mengubah arah. Hal tersebut juga berlaku untuk fungsi yang sedang turun, fungsi yang sedang turun tidak akan mencapai nilai-y yang lebih besar atau sama dengan nilai-y yang sudah dilewati tanpa mengubah arah. Perubahan arah dalam penggerakan fungsi tersebut merupakan peningkatan derajat polinomial.

CODE:

```c
#include <stdio.h>

int main(){
	int cases;
	scanf("%d\n", &cases);
	int nums[cases];
	for(int i=0; i<cases; i++){
		scanf("%d", &nums[i]);
	}
	int number = nums[0];
	int count=0; int direction = 0; //1 for increasing, -1 for decreasing
	for(int i=1; i<cases; i++){
		if(direction == 0){
			if(number > nums[i]) {
				count = i;
				direction = -1; //Decreasing
			}
			else if(number < nums[i]){
				count = i;
				direction = 1; //Increasing
			}
		}
		else if(direction == 1 && number >= nums[i]){
			count++; direction = -1;
		}
		else if(direction == -1 && number <= nums[i]){
			count++; direction = 1;
		}
		number = nums[i];
	}
	if(direction == 0) printf("0\n");
	else printf("%d\n", count);
	return 0;
}
```

Bagian Pertama

```c
int cases;
scanf("%d\n", &cases);
int nums[cases];
```

```
for(int i=0; i<cases; i++){
	scanf("%d", &nums[i]);
}
```

For-loop yang digunakan untuk mendapatkan input dari user. Bagian kode ini juga mendeklarasi 2 variabel yaitu cases dan nums. Cases adalah variabel integer yang hanya menyimpan satu nilai yaitu banyaknya nilai-y yang akan digunakan. Nums adalah variabel integer array yang akan menyimpan nilai-nilai-y tersebut. **Dengan asumsi bahwa fungsi scanf() mempunyai Time Complexity O(1), maka bagian kode ini mempunyai Time Complexity O(1) dan Space Complexity O(n).**

Bagian Kedua

```
int number = nums[0];
int count=0; int direction = 0; //1 for increasing, -1 for decreasing
for(int i=1; i<cases; i++){
	if(direction == 0){
		if(number > nums[i]) {
			count = i;
			direction = -1; //Decreasing
		}
		else if(number < nums[i]){
			count = i;
			direction = 1; //Increasing
		}
	}
	else if(direction == 1 && number >= nums[i]){
		count++; direction = -1;
	}
	else if(direction == -1 && number <= nums[i]){
		count++; direction = 1;
	}
	number = nums[i];
}
```

Bagian kedua kode ini merupakan bagian inti kodenya. Bagian kode ini terdapat 3 variabel baru: number, count, dan direction. Variabel number bertujuan untuk menyimpan nilai-y yang digunakan untuk perbandingan dengan nilai-y berikutnya. Count adalah variabel yang digunakan untuk menghitung pangkat polinomial yang paling kecil yang dapat digunakan. Direction adalah variabel yang menyimpan arah fungsi: naik (1), turun (0), atau tidak naik atau turn (0). For-loop akan berjalan untuk n (cases) kali dengan isi yang mempunyai Time Complexity O(1). **Bagian kode ini mempunyai Time Complexity O(n) dan Space Complexity O(1).**

Bagian Ketiga

```
if(direction == 0) printf("0\n");
else printf("%d\n", count);
return 0;
```

Print hasil. **Bagian kode ini mempunyai Time Complexity O(1) dan Space Complexity O(1).**

Seluruh kode ini mempunyai Time Complexity O(n) dan Space Complexity O(n)

Penjelasan kode:
Kode ini menggunakan logika matematika dimana suatu fungsi yang sedang naik atau turun tidak akan mempunyai nilai-y yang sama secara berturut turut tanpa mengubah arah. Saat melakukan analisa kembali, kode tersebut dapat diubah sehingga hanya menggunakan Space Complexity O(1). Kode tersebut akan terlihat seperti yang di bawah.

```
#include <stdio.h>

int main(){
    int cases;
    scanf("%d", &cases);
    int current, next;
    int count = 0, direction = 0;
    for(int i=0; i<cases; i++){
        if(i==0) scanf("%d", &current);
        else{
            scanf("%d", &next);
            if(direction == 0){
                if(current > next){
                    count = i;
                    direction = -1;
                }
                else if(current < next){
                    count = i;
                    direction = 1;
                }
            }
            else if (direction == 1 && current >= next){
                count++; direction = -1;
            }
            else if (direction == -1 && current <= next){
                count++; direction = 1;
            }
            current = next;
        }
    }
```

```
    printf("%d\n", count);
    return 0;
}
```

Kode tersebut mempunyai Time Complexity O(n) dan Space Complexity O(1).